

Flight Delay Big Data Analysis - Final Report

Luca Foresti

2026-05-22

Contents

1	Executive Summary	2
2	Assignment Coverage	3
2.1	Requirement-To-Output Mapping	3
3	Dataset And Preparation	4
3.1	Canonical Fields	4
3.2	Cleaning Policy	4
3.3	Generated Input Sizes	5
4	Analyses And Implementations	5
4.1	Assignment Analysis 3.2 - Delay Report by Airport and Time Period	5
4.2	Assignment Analysis 3.3 - Ranking of Airline-Airport Pairs	6
4.3	Spark SQL	7
4.4	Spark Core	7
4.5	Hive	8
4.6	MapReduce Stretch	8
5	Docker Standalone Simulation	8
6	Environment And Runtime Configuration	9
7	Benchmark Evidence	9
7.1	Spark SQL Optimization Check	10
7.2	Benchmark Pivot	11
7.3	Rows Per Second	11
7.4	Speedup Ratios	11
7.5	Normalized Scalability	11
7.6	Benchmark Charts	11
8	AWS EMR Cluster Experiment	15
8.1	Cluster Size Variation	16
9	Critical Discussion	16
10	Reproducibility And Validation	18
11	Limitations	18
12	Conclusion	18

13 Evidence Appendix: First 10 Result Rows	19
13.1 Spark SQL Samples	19
13.1.1 Delay Metrics	19
13.1.2 Delay Top-Three Causes	19
13.1.3 All-Positive Cause Companion	20
13.1.4 Ranking Metrics	20
13.1.5 Ranking Comparison	20
13.2 Spark Core Samples	20
13.2.1 Delay Metrics	20
13.2.2 Delay Top-Three Causes	21
13.2.3 Ranking Metrics	21
13.2.4 Ranking Comparison	21
13.3 Hive Samples	22
13.3.1 Delay Metrics	22
13.3.2 Delay Top-Three Causes	22
13.3.3 Ranking Metrics	22
13.3.4 Ranking Comparison	23
13.4 MapReduce Stretch Samples	23

1 Executive Summary

This project analyzes the 2024 United States flight delay dataset with a reproducible big-data workflow. The submitted artifacts show a canonical cleaned Parquet dataset, two analytical analyses implemented with three required big-data technologies plus an optional MapReduce stretch, validated outputs, benchmark evidence, and the tables and figures included in this PDF. The evidence set includes real Amazon EMR cluster execution in addition to local execution and Docker standalone simulation, reducing the earlier scalability-evidence risk while leaving the benchmark limits stated explicitly.

Repository: <https://github.com/Forest904/flight-delay-big-data-analysis.git>

The implemented analyses map directly to the assignment sections:

- **Assignment Analysis 3.2 - Delay Report by Airport and Time Period:** delay report by departure airport, month, and departure-delay range.
- **Assignment Analysis 3.3 - Ranking of Airline-Airport Pairs:** airline-airport ranking by average departure delay.

The three selected required technologies are Spark SQL, Spark Core RDDs, and Hive. Spark SQL is used as the correctness reference because its declarative aggregations and window functions make the required output rules easiest to review. Spark Core reimplements the same logic with lower-level RDD transformations. Hive provides the required SQL-on-Hadoop comparison point. Hadoop Streaming MapReduce is included as a validated stretch extension, kept separate from the grade-critical default workflow.

The main reproducibility commands are:

```
make setup
make check-env
make prepare
make generate-sizes
make run-spark-sql
make run-spark-core
make run-hive
make run-mapreduce
make benchmark-local
make benchmark-mapreduce-local
make benchmark-docker-simulation
```

```

make aws-upload
make benchmark-aws-emr
make aws-fetch-results
make charts
make report

```

The `make benchmark-docker-simulation` target runs single-host Docker standalone simulation evidence. The AWS targets reproduce the EMR upload, benchmark, and result-fetch workflow when AWS Academy Learner Lab access and budget are available; the report uses the completed EMR runs recorded under `experiments/results/aws-emr/` and `experiments/results/aws-emr-larger/`.

2 Assignment Coverage

This report covers the required elements as follows:

- Data preparation: Dataset And Preparation.
- Implementation choices and pseudocode: Analyses And Implementations.
- First 10 result rows: Evidence Appendix.
- Execution-time tables and charts: Benchmark Evidence and AWS EMR Cluster Experiment.
- Execution settings: Docker Standalone Simulation, Environment And Runtime Configuration, and AWS EMR Cluster Experiment.
- Critical discussion: Critical Discussion.
- GitHub repository link: Executive Summary.

2.1 Requirement-To-Output Mapping

The two selected analyses use stable output schemas so every assignment requirement can be traced to concrete columns in the generated CSV files and first-10 report samples.

Analysis 3.2 requirement	Output column(s)
Departure airport	<code>origin_airport</code>
Month	<code>month</code>
Departure-delay range	<code>delay_range</code>
Number of flights	<code>flight_count</code>
Average departure delay	<code>avg_departure_delay</code>
Average arrival delay	<code>avg_arrival_delay</code>
Top cause labels	<code>top_1_cause</code> , <code>top_2_cause</code> , <code>top_3_cause</code>
Top cause counts	<code>top_1_count</code> , <code>top_2_count</code> , <code>top_3_count</code>
All positive causes companion	<code>delay_by_airport_month_all_causes: cause_rank</code> , <code>cause</code> , <code>cause_count</code>

Analysis 3.3 requirement	Output column(s)
Departure airport	<code>origin_airport</code>
Airline	<code>airline</code>
Number of flights	<code>flight_count</code>
Average departure delay	<code>avg_departure_delay</code>
Average arrival delay	<code>avg_arrival_delay</code>
Cancellation rate	<code>cancellation_rate</code>
Airport average delay	<code>airport_avg_departure_delay</code>
Difference from airport average	see note below
Airline rank at airport	<code>rank_at_airport</code>

The `airline` output field is the normalized `airline_code`. The raw dataset does not provide a stable airline-name field, so code-based airline identity is used consistently across Spark SQL, Spark Core, Hive, and the MapReduce stretch.

The full CSV column for the difference from the airport average is `difference_from_airport_avg_departure_delay`; the shortened table label above keeps the PDF readable without changing the generated output schema.

3 Dataset And Preparation

The source dataset is Kaggle `hrishitpatil/flight-data-2024`, stored locally as `data/raw/flight_data_2024.csv`. The raw CSV has 7,079,081 rows, 35 columns, and a local file size of 1,309,010,752 bytes. The raw file is not committed to Git because it is large and externally hosted.

The preparation pipeline is implemented by `src/preparation/prepare_spark.py` and is run with:

```
make prepare
```

The prepared output is `data/prepared/flights_2024_clean.parquet`. This Parquet dataset is the canonical input for Spark SQL, Spark Core, and Hive. MapReduce uses a canonical CSV export derived from the same prepared Parquet input, not the raw Kaggle CSV. Using one prepared source keeps the comparison fair: each implementation receives the same normalized schema, the same row-retention policy, and the same benchmark input files.

3.1 Canonical Fields

The prepared dataset keeps only the fields needed by the selected analyses:

- Date and grouping: `flight_date`, `month`.
- Airline fields: `airline_code`, plus nullable `airline_name` because the raw dataset has no stable airline-name field.
- Airport fields: `origin_airport`, `destination_airport`.
- Delay metrics: `departure_delay`, `arrival_delay`.
- Cancellation and diversion: `cancelled`, `diverted`, `cancellation_code`.
- Delay causes: carrier, weather, NAS, security, and late-aircraft delay minutes.

3.2 Cleaning Policy

The cleaning policy is conservative. Rows are removed only when structural fields are invalid after casting: missing or invalid flight date, month, airline code, origin airport, destination airport, cancelled flag, or diverted flag. Cancelled flights, diverted flights, null delay values, and negative delay values are preserved.

Empty strings are normalized to null. Numeric delay fields are cast to numeric types but are not imputed. This matters because Spark and Hive averages ignore null values, while cancellation-rate calculations still need cancelled flights in the denominator.

The raw data inspection command `.\venv\Scripts\python.exe scripts\inspect_raw_dataset.py` found 92,970 null departure-delay rows and 113,814 null arrival-delay rows. A read-only cross-count over the same raw CSV found that all 92,970 null departure-delay rows are cancelled rows with a cancellation code. These rows are retained in the prepared dataset unless they fail a structural rule. Negative delay values are preserved because they represent early departures or arrivals, not data errors.

Raw audit measure	Rows
Total raw rows	7,079,081
Rows with null <code>departure_delay</code> / raw <code>dep_delay</code>	92,970
Cancelled rows	96,315
Cancelled rows with null <code>departure_delay</code>	92,970

Raw audit measure	Rows
Cancelled rows with null <code>departure_delay</code> and a cancellation code	92,970

3.3 Generated Input Sizes

The input-size generator creates controlled benchmark inputs under `data/generated/`:

- 100k
- 500k
- 1m
- 3m
- full
- 14m
- 28m
- 1m_hc8

Smaller inputs are selected with a deterministic hash-based method using seed 20240520. This avoids chronological bias from simply taking the first rows of the CSV. The 14m and 28m inputs were generated and validated with controlled replication. The 14m input uses one complete repetition of the canonical prepared dataset plus a deterministic sampled remainder of 6,920,919 rows. The 28m input uses three complete repetitions plus a deterministic sampled remainder of 6,762,757 rows. Both outputs keep the same prepared schema, and no synthetic columns or schema changes are retained.

These replicated larger inputs are used as scalability stress evidence because they increase data volume while preserving the canonical row shape, column types, null behavior, and analysis semantics used by the original prepared dataset. They are not treated as new statistical observations about flight behavior. In particular, 14m and 28m test how the implementations respond to larger input volume under the controlled benchmark setup.

The optional 1m_hc8 input is a separate synthetic cardinality stress input. It keeps the 1m row count, prepared schema, numeric delay values, cancellation fields, and delay-cause values unchanged, but appends one of eight deterministic suffixes (`_HC00` through `_HC07`) to `origin_airport`, `airline_code`, and non-null `airline_name`. This increases grouping-key and aggregate-output cardinality without claiming new flight observations. Report tables identify it as `input_kind = high_cardinality_stress`, and any 1m_hc8 benchmark rows are interpreted only as shuffle/cardinality stress evidence.

4 Analyses And Implementations

4.1 Assignment Analysis 3.2 - Delay Report by Airport and Time Period

This job implements the assignment requirement to report delay behavior for each departure airport and time period. In this submission the time period is the flight month. It groups rows by:

- `origin_airport`
- `month`
- `delay_range`

The delay ranges are:

- low: `departure_delay < 15`
- medium: `15 <= departure_delay <= 60`
- high: `departure_delay > 60`

Rows with known departure delay are assigned only to these three numeric ranges. Cancelled rows with null departure delay cannot be placed in a numeric delay range, so the analysis reports them separately with `delay_range = cancelled_no_departure_delay`. In the raw audit above, that accounts for the 92,970

cancelled rows whose departure delay is null but whose cancellation code is available. Other rows with null departure delay remain outside this analysis. For each group, the output reports flight count, average departure delay, average arrival delay, and the three most frequent delay or cancellation causes.

The stable output schema includes:

Field group	Columns
Grouping keys	<code>origin_airport</code> , <code>month</code> , <code>delay_range</code>
Metrics	<code>flight_count</code> , <code>avg_departure_delay</code> , <code>avg_arrival_delay</code>
Top causes	<code>top_1_cause</code> , <code>top_1_count</code> , <code>top_2_cause</code> , <code>top_2_count</code> , <code>top_3_cause</code> , <code>top_3_count</code>

Cause labels in the core `delay_by_airport_month` output are dominant causes: one cause is selected per flight, using cancellation codes first and otherwise the largest positive delay-cause field. In the `cancelled_no_departure_delay` bucket, cancellation codes are reported as `cancellation:<code>`, with `cancellation:unknown` as a fallback if a cancelled row has no code. Cause ties are deterministic: cause count descending, then cause label ascending. Groups with fewer than three available causes use an empty cause value and count 0.

The companion `delay_by_airport_month_all_causes` output removes the ambiguity in the assignment wording by counting every positive delay-cause field: `carrier_delay`, `weather_delay`, `nas_delay`, `security_delay`, and `late_aircraft_delay`. Cancelled flights also contribute `cancellation:<code>` when a cancellation code is available. Flights with no positive delay-cause field and no available cancellation code do not appear in the companion view. The companion schema is normalized:

Field group	Columns
Grouping keys	<code>origin_airport</code> , <code>month</code> , <code>delay_range</code>
Cause ranking	<code>cause_rank</code> , <code>cause</code> , <code>cause_count</code>

Textual pseudocode:

```
keep rows with known departure_delay or cancelled rows with null departure_delay
derive numeric delay_range from known departure_delay
derive cancelled_no_departure_delay for cancelled rows without departure_delay
derive cause from cancellation code or largest positive delay-cause field
group by origin_airport, month, delay_range
compute flight count and average delays
count causes inside each group
rank causes by count descending and cause label ascending
emit the first three causes and pad missing slots with count 0
order output deterministically
```

All-positive companion pseudocode:

```
keep the same rows as the dominant-cause delay analysis
emit one event for each positive delay-cause field
emit cancellation:<code> for cancelled rows with an available code
group by origin_airport, month, delay_range, cause
rank causes by count descending and cause label ascending
emit normalized cause_rank, cause, and cause_count rows
```

4.2 Assignment Analysis 3.3 - Ranking of Airline-Airport Pairs

This job implements the assignment requirement to compare each airline-airport pair against the airport average. For each (`origin_airport`, `airline_code`) pair it computes:

- flight count;
- average departure delay;
- average arrival delay;
- cancellation rate, using all flights in the pair as the denominator, including cancelled flights;
- airport average departure delay;
- difference from the airport average;
- rank at the airport, from lowest average departure delay to highest.

Textual pseudocode:

```
group flights by origin_airport and airline_code
compute airline-level counts, averages, and cancellation rate
group flights by origin_airport
compute airport-level average departure delay
join airline metrics with airport metrics
rank airlines per airport by avg_departure_delay ascending, nulls last
order output by airport, rank, average delay, and airline code
```

4.3 Spark SQL

Spark SQL is the reference implementation. It expresses both analyses with DataFrame temporary views, SQL aggregations, and window functions. The ranking job uses `RANK() OVER (PARTITION BY origin_airport ORDER BY avg_departure_delay ASC NULLS LAST)`. The delay job uses grouped cause counts and compact per-group cause arrays to select the top three causes with deterministic tie-breaking.

Spark SQL is the most concise implementation in this project. The grouping keys, derived fields, ranking rule, and top-three cause rule are visible in the query structure. The cost is that grouped aggregations and window functions still require shuffles; declarative syntax makes the logic easier to maintain, but it does not remove distributed execution cost.

Execution plan: `make run-spark-sql` reads the canonical prepared Parquet input with the Spark settings from `config/local.yaml`, registers a temporary view, executes one SQL query per analysis, collects each ordered aggregate result once, writes a headered `full/part-00000.csv` plus first-10 sample under `outputs/spark_sql/`, and records runtime metrics in `outputs/spark_sql/runtime_metrics.json`. Its outputs are validated directly and then used as the correctness reference for Spark Core, Hive, and the MapReduce stretch.

4.4 Spark Core

Spark Core reimplements the Spark SQL logic with RDD transformations. It uses `reduceByKey` for delay aggregates, cause counts, airline aggregates, and airport aggregates. This avoids `groupByKey` for large aggregations and allows partial accumulator state to be combined before the shuffle.

The Spark Core implementation is intentionally explicit. It builds accumulator objects for counts, sums, cancellation totals, and cause counts; joins compact aggregate RDDs by airport; and sorts per-airport airline rows to reproduce the Spark SQL rank semantics. This gives more control over the physical steps but also makes the implementation more verbose and easier to get wrong.

Execution plan: `make run-spark-core` reads the same prepared Parquet input, converts rows into RDD records, runs the two analysis pipelines with keyed aggregations, joins, and deterministic sorts, then materializes the small result tables to the same CSV contract used by Spark SQL. Runtime metrics are written under `outputs/spark_core/`, and validation compares Spark Core row counts, keys, numeric values, top-three causes, and ranking order against Spark SQL.

4.5 Hive

Hive is the third required technology. The runner starts the Docker Compose Hive stack, creates an external table over the prepared Parquet data, executes HiveQL versions of both analyses, and exports CSV outputs with the same schemas as Spark SQL and Spark Core.

Hive is useful as a SQL-on-Hadoop comparison point. Its SQL syntax is familiar for aggregation-heavy work, but this project runs Hive as a local containerized HiveServer2/metastore/PostgreSQL stack. The setup does not use a Hive-on-YARN or HDFS service, so the Hive results should be read as controlled local/container evidence rather than as managed-service Hive performance.

Execution plan: `make run-hive` starts the local Docker Compose Hive services, creates an external Hive table over the prepared Parquet directory, runs the two HiveQL scripts, exports query results through temporary text tables, and normalizes them into the shared CSV output layout under `outputs/hive/`. Runtime metrics record per-query duration and output rows. Validation compares the exported Hive outputs against the Spark SQL reference.

4.6 MapReduce Stretch

Hadoop Streaming MapReduce is implemented as an optional stretch, not as part of the three required technology baseline. The implementation uses Python mappers and reducers under `src/mapreduce/`, runs through a Docker-based `mapreduce-runner` service, and writes the same output schemas under `outputs/mapreduce/`.

MapReduce reads a canonical CSV export generated from prepared Parquet. The export is stored under `data/generated/mapreduce_csv/` with a manifest that records the source Parquet signature, row count, CSV size, CSV SHA-256, and export duration. This keeps the stretch comparison fair without reparsing the raw Kaggle CSV, while avoiding stale cached exports.

The delay reducer computes the same grouped counts, averages, and top-three cause ordering as Spark SQL. The ranking reducer groups by airport, aggregates airline statistics, computes the airport average departure delay, and applies Spark SQL `RANK()` semantics with null averages last. Validation passed against Spark SQL for the full prepared dataset; the separate benchmark campaign uses the `100k`, `500k`, `1m`, `3m`, and `full` inputs as optional stretch evidence.

The supported runtime is Docker-local Hadoop Streaming, not YARN/HDFS. MapReduce benchmark runs use isolated output roots under `outputs/mapreduce/.benchmark_runs/`, leaving `outputs/mapreduce/` reserved for validated report-ready outputs.

Execution plan: `make run-mapreduce` exports the selected prepared Parquet input to a canonical CSV with a manifest, runs Hadoop Streaming mapper/reducer jobs inside the Docker-local runtime, sorts and normalizes reducer output into the shared CSV schemas, and records runtime metrics under `outputs/mapreduce/`. The validator compares the stretch outputs against Spark SQL before the report uses them as evidence.

5 Docker Standalone Simulation

The Docker standalone simulation is the repository's controlled alternative execution setting. Spark uses Docker Compose with one Spark standalone master, two Spark workers, and a driver service. The workers are separate containers and processes, but they all run on one physical laptop and share Docker Desktop CPU, memory, and disk limits. Data is read from a bind-mounted project directory, not HDFS or object storage. This is not a true distributed cluster; it is a repeatable single-host simulation of Spark standalone execution.

Hive is included in the Docker benchmark CSV so the report contains rows for all three required technologies, but Hive remains a single-node containerized Hive setup. Therefore the report uses the phrase "Docker standalone simulation" and avoids presenting these runs as managed-service performance. The `3m`, `full`, `14m`, and `28m` Docker inputs were configured as Sprint P2 stretch targets and completed successfully for Spark SQL, Spark Core, and Hive with three repetitions per technology/input cell.

The documented topology is stored in `docs/docker_simulation.md`. The command used to run the simulation evidence is:

```
make benchmark-docker-simulation
```

6 Environment And Runtime Configuration

The environment summary was generated by `scripts/generate_environment_summary.py`. The complete artifact is stored at `report/tables/environment_summary.md`.

Category	Item	Value
Host OS	Windows detail	Microsoft Windows 11 Home 10.0.26200 build 26200 64-bit
CPU	Model	AMD Ryzen AI 7 350 w/ Radeon 860M
CPU	Cores	8 physical, 16 logical
Memory	Host RAM	33,598,853,120 bytes
Disk	Project drive	NVMe SSD, 1,024,209,543,168 bytes
Python	Version	3.12.2
Java	Version	OpenJDK 17.0.19
PySpark	Version	4.1.1
Hive	Base image	apache/hive:4.0.1
MapReduce	Base image	apache/hive:4.0.1 with Hadoop Streaming and Python 3
Docker	Version	29.4.3
Docker Compose	Version	v5.1.3
Docker Desktop	Limits	16 CPUs, 16,391,536,640 memory bytes
Local Spark	Master and partitions	local[*], 8 shuffle partitions
Docker Spark	Master and partitions	spark://spark-master:7077, 16 shuffle partitions
Docker topology	Spark	1 master, 2 workers, 2 cores and 2 GB memory per worker
Docker topology	Hive	HiveServer2, metastore, PostgreSQL, single host

7 Benchmark Evidence

The benchmark runner records technology, job name, input size, environment, execution-setting label, repetition number, raw duration, output rows, status, timestamp, input path, metrics path, and phase timings when the runner exposes them. By default, each selected input/technology configuration is run three times. Timestamped benchmark CSVs remain raw audit evidence, while the report-ready artifacts under `report/tables/` and `report/figures/` aggregate successful repetitions.

The reporting pipeline computes median, mean, minimum, maximum, and standard deviation of duration for each environment/input/job/technology group. `report/tables/benchmark_summary.md` is the detailed execution-time table: it shows `runs`, median, mean, min, max, standard deviation, output rows, run ID, and timestamp. `report/tables/benchmark_phase_summary.md` records median phase timings for rows that expose them. Single-run CSVs are still readable as `runs=1`, and fresh benchmark campaigns can be forced to one run for smoke checks with `BENCHMARK_FLAGS="--repetitions 1"`. A `runs=1` row is not treated as statistically equivalent to a repeated campaign; `report/tables/benchmark_notes.csv` marks those cells as `smoke`, `budget_limited`, or `resource_limited`.

The benchmark duration column is the per-analysis job duration reported by the technology runner after the input has been prepared. It covers the analysis execution and output materialization, but it is not a pure compute-only number and does not include the one-time data preparation pipeline. Spark SQL and Spark Core use the same small-result materialization boundary: collect the ordered aggregate once to the driver, write `full/part-00000.csv`, and derive `first_10.csv` from the same rows. For those runners, `result_collect_seconds` is the main Spark action boundary because lazy evaluation makes transformation, aggregation, ordering, and driver collection inseparable without adding extra actions. Some fixed costs still

shape interpretation: Spark/JVM startup, Docker container and service startup, Hive service startup, query planning, local file listing, S3 I/O, and EMR step scheduling can dominate the shorter runs or appear in surrounding step/command timing artifacts rather than inside the per-job aggregate alone. Local and Docker runs read prepared Parquet from a warm local filesystem or bind mount, so OS cache and columnar Parquet reads affect the observed throughput. EMR runs read and write through S3 and are scheduled as managed Spark steps, so object-store and scheduler overhead are part of the managed execution path.

The opt-in MapReduce stretch benchmark is reported separately from the required default benchmark matrix:

- Local required technologies: 100k, 500k, 1m, 3m, full, 14m, and 28m; Spark SQL, Spark Core, and Hive; both jobs; 42/42 successful with three repetitions each.
- Docker standalone simulation: 100k, 500k, 1m, 3m, full, 14m, and 28m; Spark SQL, Spark Core, and Hive; both jobs; 42/42 successful with three repetitions each.
- AWS EMR baseline cluster: 100k, 500k, 1m, 3m, full, and 14m; Spark SQL and Spark Core; both jobs; 24/24 successful. The 500k, 3m, and 14m cells were strengthened to two repetitions.
- AWS EMR larger cluster: 1m, full, 14m, and 28m; Spark SQL and Spark Core; both jobs; 16/16 successful. The 28m cells are single-run smoke/stress evidence.
- Local MapReduce stretch: 100k, 500k, 1m, 3m, and full; Hadoop Streaming MapReduce; both jobs; 10/10 successful with three repetitions each.

The local required-technology run IDs are 20260522T133525179005Z, 20260522T142052973274Z, and 20260522T144129099690Z. Docker standalone simulation uses the original 100k/500k/1m run 20260522T161751426398Z plus Sprint P2 large-input runs from 20260523T002216937593Z through 20260523T010842627918Z. The MapReduce stretch benchmark run ID is 20260522T182250129354Z. AWS EMR uses m4-emr-final-2 and m4-hardened-smoke-3 for the earlier baseline/smoke evidence, then m4-emr-p2-weak-cells, m5-emr-p2-14m, and m5-emr-p2-28m-smoke for Sprint P2. AWS 500k, 3m, and 14m baseline cells now have two repetitions, larger-profile 14m has three repetitions, and larger-profile 28m is retained as single-run smoke/stress evidence. EMR Hive was explicitly excluded from Sprint P2, so this report does not claim Hive cluster evidence. The full status matrix is stored in `report/tables/benchmark_status.md`; cells that were not run remain explicit N/A or `not_run` entries rather than inferred values.

The full generated table in `report/tables/benchmark_summary.md` contains the same statistics for every successful environment/input/job/technology group. Compact examples from that table are:

- Local 28m delay, Spark SQL: 3 runs; median 11.924 s; min 11.790 s; max 11.973 s.
- Local 28m delay, Spark Core: 3 runs; median 1.333 s; min 1.252 s; max 1.625 s.
- Local 28m delay, Hive: 3 runs; median 101.265 s; min 100.484 s; max 103.605 s.
- Docker 28m delay, Spark SQL: 3 runs; median 14.397 s; min 14.270 s; max 15.525 s.
- AWS EMR baseline 14m delay, Spark SQL: 2 runs; median 17.020 s; min 16.676 s; max 17.363 s.
- AWS EMR larger 28m delay, Spark SQL: 1 smoke/stress run; 19.184 s.
- AWS EMR 100k delay, Spark SQL: 1 smoke run; 15.832 s.

7.1 Spark SQL Optimization Check

Milestone 4 adds a targeted Spark SQL before/after benchmark lane for 1m, full, and 14m in local and Docker standalone simulation environments. The raw M4 rows use distinct labels, such as `local-spark-sql-baseline-m4` and `local-spark-sql-optimized-m4`, so optimized evidence does not silently replace the main baseline matrix. The generated comparison table is `report/tables/spark_sql_optimization_befo`

The optimization keeps the schemas and deterministic output order but changes Spark SQL materialization to match Spark Core's small-result path: collect the ordered aggregate once, write `full/part-00000.csv`, and derive `first_10.csv` from the same rows. The delay-cause ranking also avoids a separate top-three window by ranking compact grouped cause lists. The measured local result is mixed but mostly positive: 7 of 9 cells improved, including `full delay_by_airport_month` from 7.842 s to 6.812 s, while `full delay_by_airport_month_all_causes` moved from 5.053 s to 5.328 s. Docker standalone simulation improved all 9 M4 cells, including `14m delay_by_airport_month` from 18.714 s to 9.469 s and `14m`

`delay_by_airport_month_all_causes` from 13.169 s to 6.900 s. The remaining physical plans still include scans, exchanges, aggregations, sorts, and the airline ranking window, so SQL expressiveness does not remove shuffle or sort costs.

7.2 Benchmark Pivot

For compactness, `report/tables/benchmark_pivot.md` focuses on median duration by technology. It is useful for scanning, but the benchmark-summary table above is the statistical source because it carries run counts, variability, and single-run context. The pivot intentionally leaves unavailable technology/input combinations blank or N/A rather than extrapolating them.

7.3 Rows Per Second

Rows-per-second values in `report/tables/rows_per_second.md` are computed as input records divided by median elapsed seconds. Throughput appears higher under several larger-input local and Docker cells, but fixed costs, caching, prepared Parquet columnar reads, and small aggregate output cardinality affect interpretation. These throughput values summarize end-to-end per-analysis duration, including output materialization, rather than isolated Spark compute. The replicated 14m and 28m rows are therefore stress tests of the execution setup, not new flight-behavior observations.

7.4 Speedup Ratios

Each value in `report/tables/speedup.md` is a median-duration ratio. A value above 1 means the numerator took longer than the denominator for the aggregate row. These ratios are descriptive for the measured setup and are not portable performance constants.

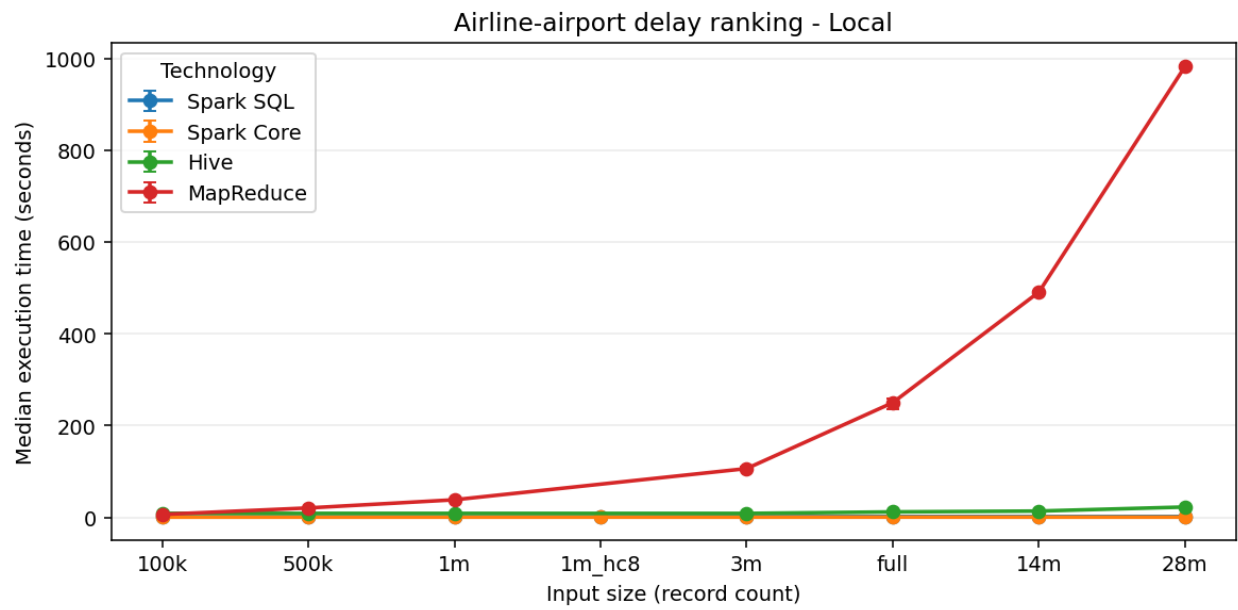
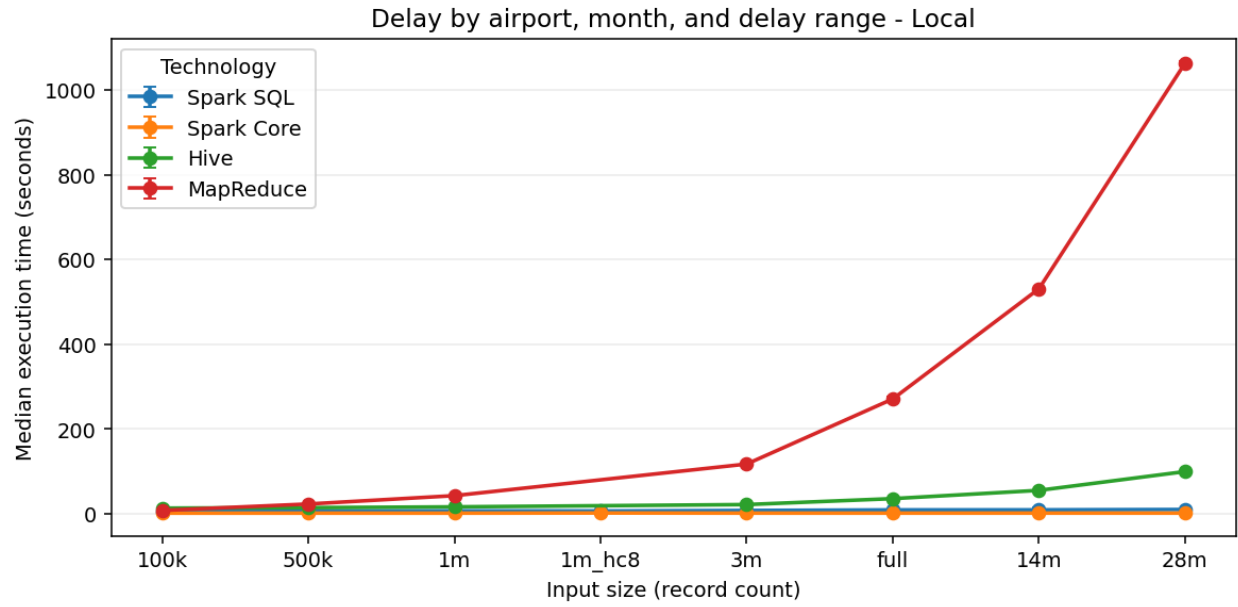
7.5 Normalized Scalability

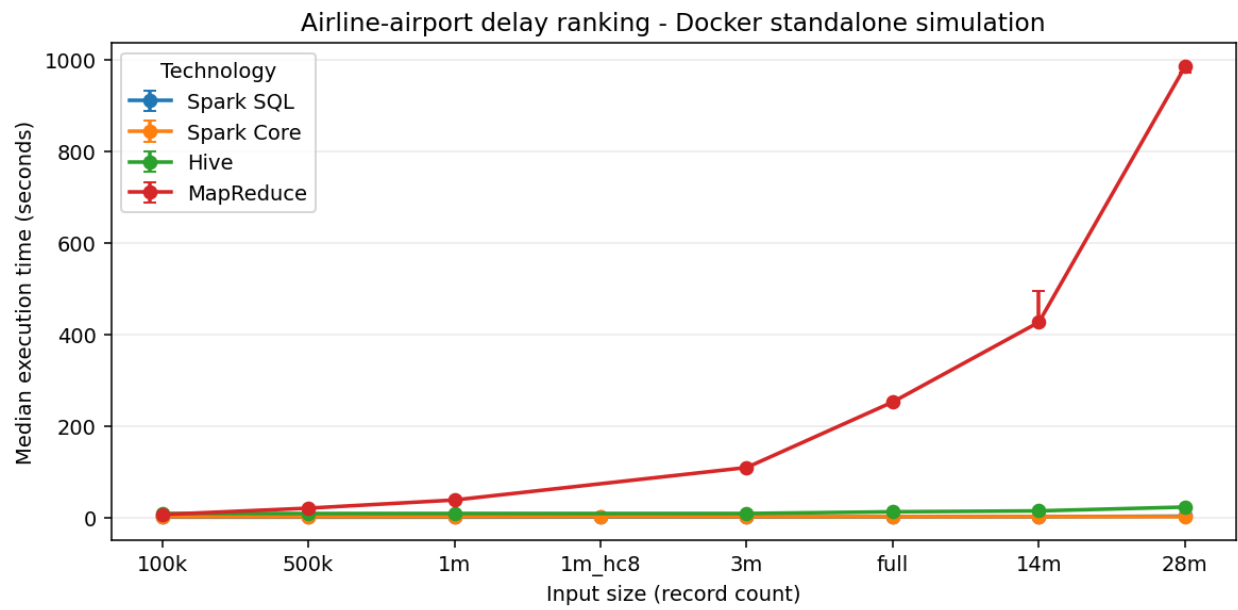
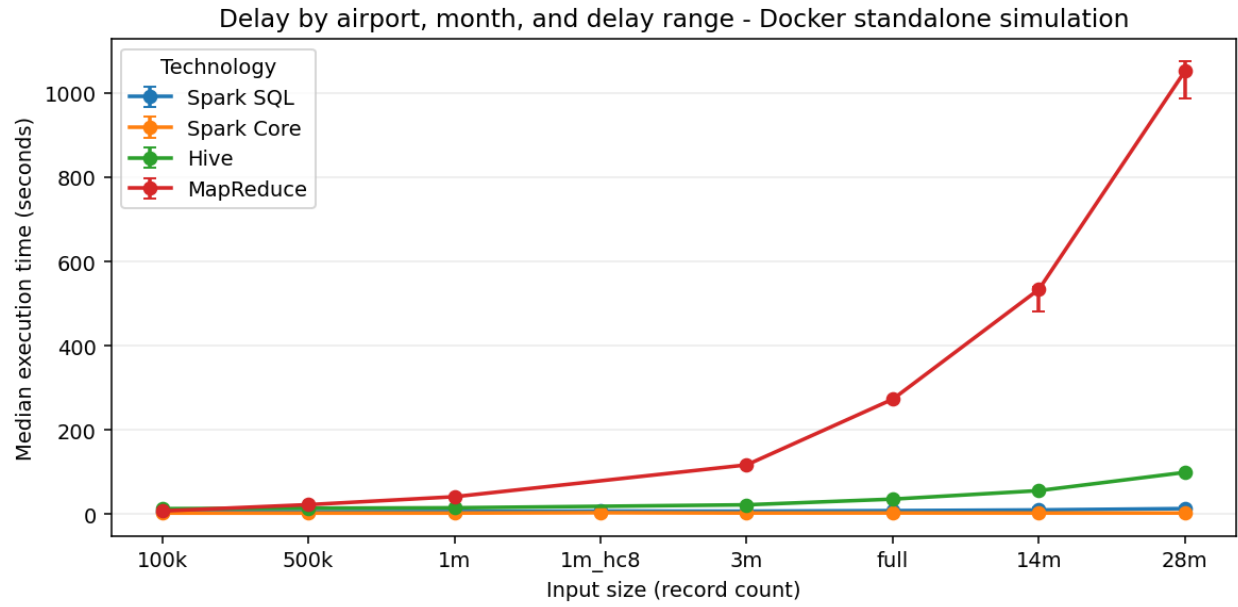
`report/tables/scalability_ratios.md` normalizes median duration, record count, and throughput against the 100k baseline for each environment, job, and technology. The ratios help describe where throughput appears higher under the measured setup, but fixed startup costs, cache behavior, shuffle and aggregation pressure, and the small result cardinality limit what can be inferred from them.

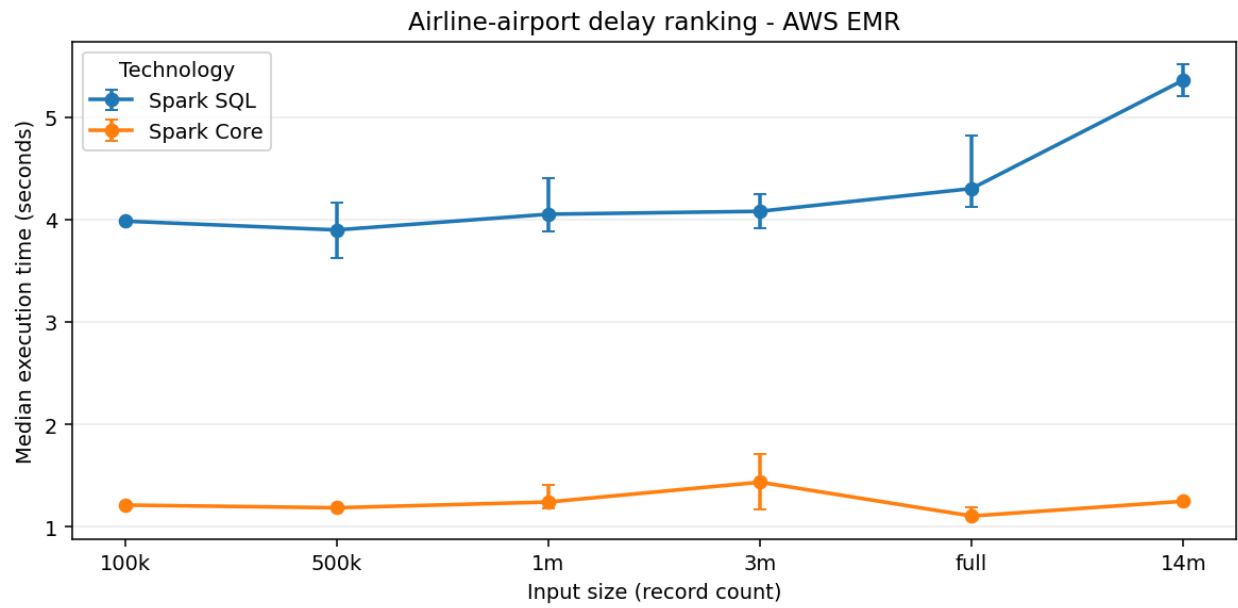
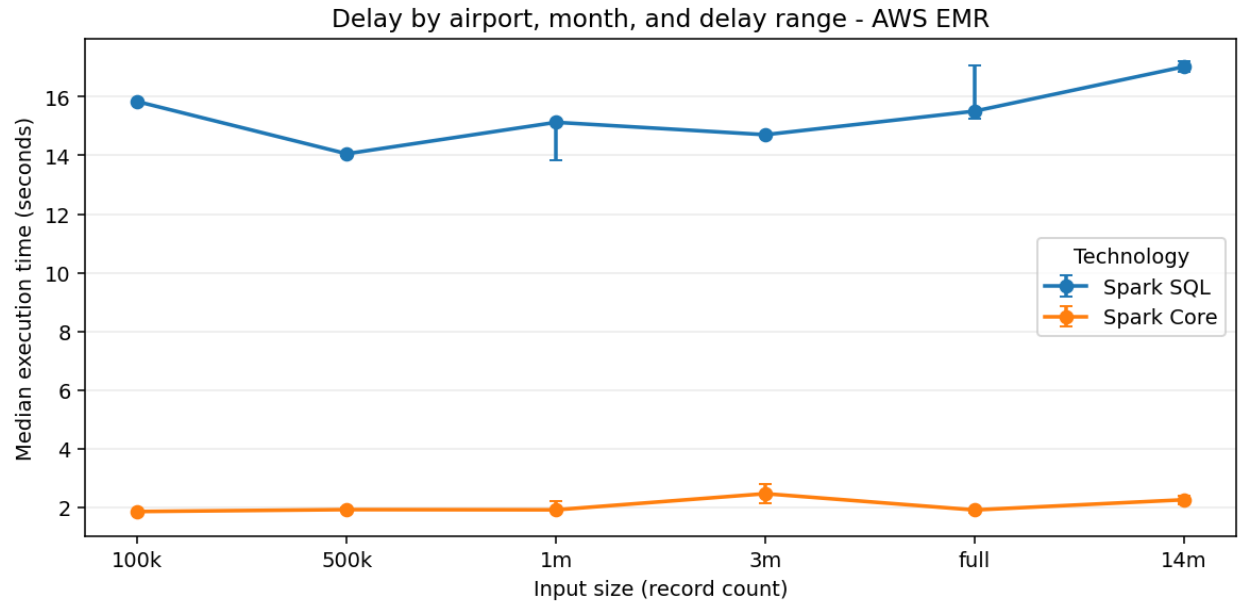
High-cardinality stress rows are kept out of this row-volume scalability table. When `1m_hc8` benchmark evidence is present, the generated `report/tables/cardinality_stress_comparison.md` table compares it directly against the `1m` baseline by environment, Spark technology, and job. That table reports both median-duration ratios and aggregate-output-row ratios, so the discussion separates two stress dimensions: more rows with mostly unchanged key cardinality (14m and 28m) versus the same row count with more grouping keys and shuffle/output partitions (`1m_hc8`).

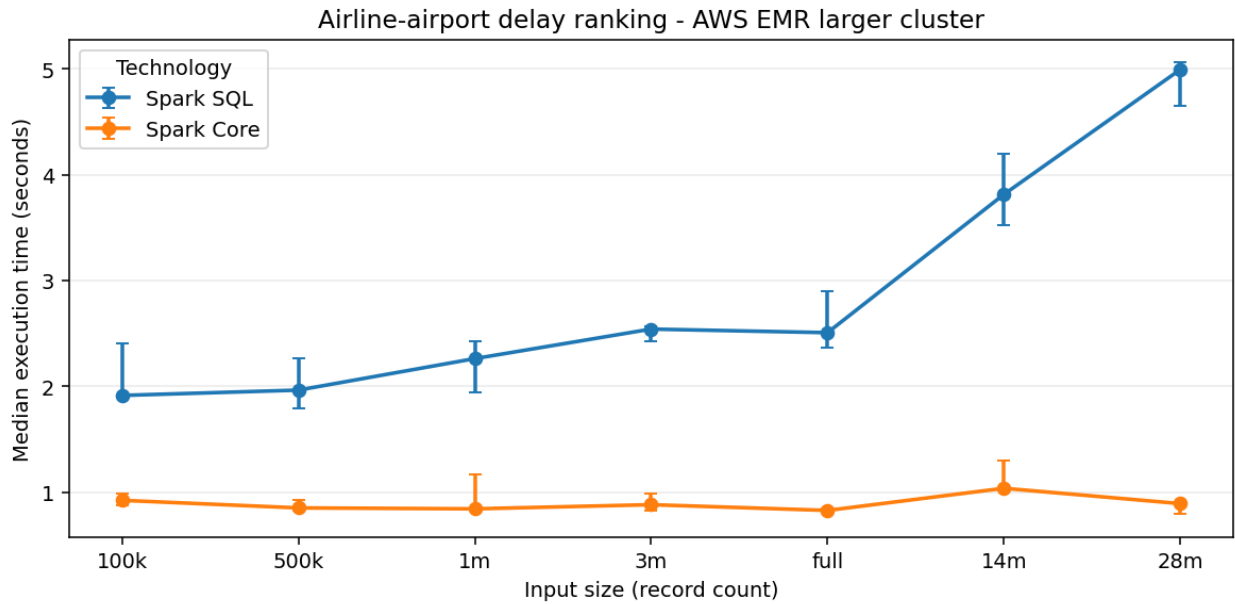
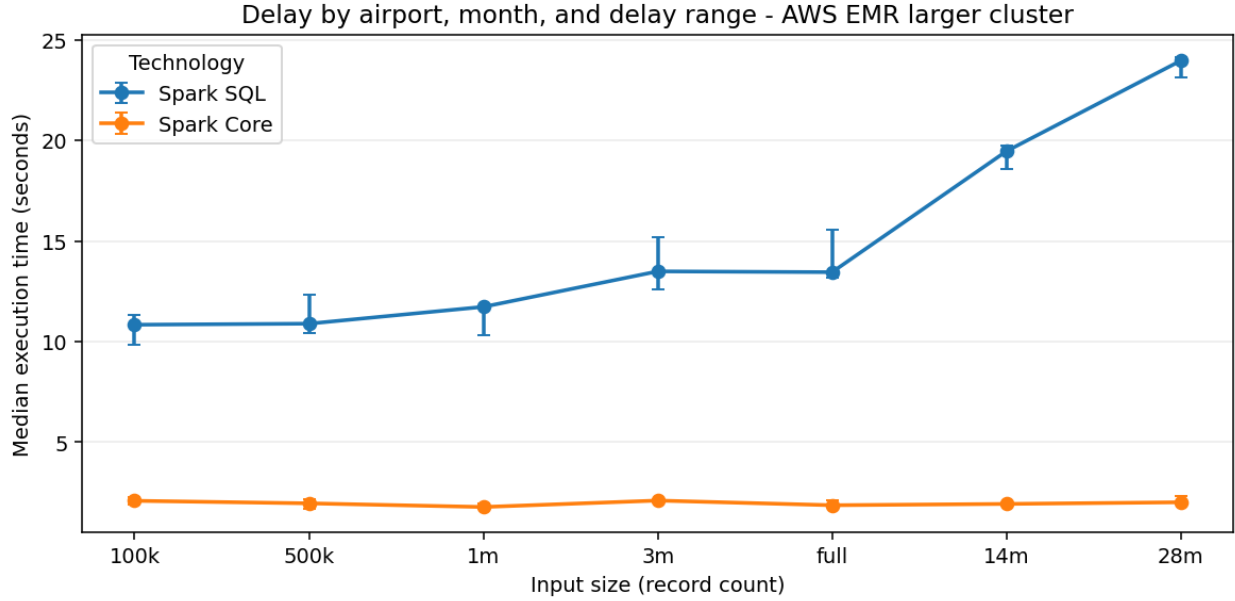
7.6 Benchmark Charts

Execution-time charts are generated separately for local execution, Docker standalone simulation, AWS EMR baseline, and AWS EMR larger-cluster runs. Charts plot median duration and include min/max variability indicators when repeated successful measurements are available. Line plots are used only when at least three input sizes are available for a job and environment. To keep the PDF readable, chart tick labels use compact input labels; the record counts are 100k = 100,000, 500k = 500,000, 1m = 1,000,000, 3m = 3,000,000, full = 7,079,081, 14m = 14,000,000, and 28m = 28,000,000.









8 AWS EMR Cluster Experiment

The project includes real Amazon EMR evidence in addition to local and Docker standalone simulation runs. The AWS lane uses Spark steps on YARN, S3 for input and output movement, and short-lived clusters created specifically for benchmark evidence. Amazon's EMR Spark documentation and the EMR 7.13.0 release notes list Spark 3.5.6-`amzn-2` for release `emr-7.13.0`: <https://docs.aws.amazon.com/emr/latest/ReleaseGuide/emr-spark.html> and <https://docs.aws.amazon.com/emr/latest/ReleaseGuide/emr-7130-release.html>.

The baseline EMR profile used `m5.xlarge` nodes with one primary and two core nodes. Its main run was `m4-emr-final-2` on cluster `j-VS60EAAXUMGP`; the supplemental baseline runs were `m4-emr-p2-weak-cells` and `m4-hardened-smoke-3`. This profile covered 100k, 500k, 1m, 3m, full, and 14m inputs for Spark SQL and Spark Core.

The larger EMR profile used the same EMR release and instance type with one primary and three core nodes.

Its main run was `m5-emr-3core-1m-full` on cluster `j-LTX1FIHYB4X9`; the supplemental larger runs were `m5-emr-p2-14m` and `m5-emr-p2-28m-smoke`. This profile covered 1m, full, 14m, and 28m inputs for Spark SQL and Spark Core.

The larger 14m and 28m inputs were run as separate P2 larger-profile clusters, not inside the canonical `m5-emr-3core-1m-full` cluster. They share the same node profile and Spark/YARN path, so the report groups them as larger EMR evidence while preserving separate run IDs and costs. The main larger run cost 0.3944 USD; the larger 14m run cost 0.3911 USD; and the larger 28m smoke run cost 0.2182 USD, as recorded in `aws_cost_log`.

The S3 layout is deliberately simple and reproducible:

- `.../flight-delay/data/`: prepared and generated Parquet inputs.
- `.../flight-delay/code/`: source bundles and per-run runtime configs.
- `.../flight-delay/results/`: runtime metrics, benchmark outputs, and fetched result evidence.
- `.../flight-delay/logs/`: EMR and Spark step logs.

The report-ready AWS evidence is generated into small audit artifacts instead of relying on console screenshots. `report/tables/aws_run_manifest.md` records cluster IDs, region, EMR release, instance type, node count, source-bundle hash, runtime-config hash, and dependency pins where available. `report/tables/aws_step_timing.md` records EMR step queue/start/end timings and S3 metrics URIs. `report/tables/aws_cost_log.md` records cluster lifetime, node-hours, configured hourly cost estimates, and Learner Lab budget fields. The aggregate timings flow into `report/tables/benchmark_summary.md`, and the local/Docker/EMR size comparison is summarized in `report/tables/cluster_size_comparison.md`.

Cost controls were built into the workflow before running the expensive steps: the AWS config targets the AWS Academy Learner Lab budget, uses short-lived clusters, validates S3 input prefixes before cluster creation, sets cluster startup, step, and total-run timeouts, enables a 600-second idle termination policy for the hardened/larger runs, terminates the tracked cluster in the benchmark helper, and exposes `make aws-cleanup` for tagged-cluster cleanup. This makes the AWS results reproducible while avoiding any claim that the experiment was an unconstrained production benchmark.

8.1 Cluster Size Variation

The cluster-size comparison uses `m4-emr-final-2` plus `m4-emr-p2-weak-cells` as the baseline-profile evidence and `m5-emr-3core-1m-full`, `m5-emr-p2-14m`, and `m5-emr-p2-28m-smoke` as the larger-profile evidence. The larger runs used one primary node and three core nodes, the same `m5.xlarge` instance type, the same EMR release, and the same Spark/YARN execution path. The P2 larger 14m run completed on cluster `j-UTYJLQJEQUZ` with an estimated cost of 0.3911 USD; the P2 larger 28m smoke run completed on cluster `j-1XB692RVA04ZI` with an estimated cost of 0.2182 USD. Docker standalone simulation is now shown for every configured input size, including full, 14m, and 28m.

The generated cluster-size comparison table is stored in `report/tables/cluster_size_comparison.md`. Compared with the baseline EMR profile, the larger profile had lower Spark Core medians in all four comparable cells, with measured speedup ratios from 1.146x to 1.221x. Spark SQL was mixed: it had a lower median for the 1m delay job (1.052x) but higher medians for 1m ranking, full delay, and full ranking (0.951x to 0.991x). This is limited scalability evidence rather than a general speedup claim: adding one core node did not uniformly accelerate these short analytical jobs, and fixed Spark startup, S3 file listing and reads, shuffle planning, and small output cardinality remain large parts of the measured runtime. The interpretation is further limited because clusters are short-lived, S3 I/O is part of the managed execution path, repetitions are constrained by budget, and Learner Lab service limits prevent wider cluster-size sweeps.

9 Critical Discussion

Spark SQL is the most expressive implementation. It is especially strong for the two selected analyses because both are naturally relational: they derive columns, group by keys, aggregate measures, and rank

rows inside partitions. Window functions make the airline ranking and top-three cause selection clear. The tradeoff is that the physical plan still has to shuffle data for grouped aggregations and window partitions. Spark SQL hides some mechanics, but it does not make those costs disappear.

Spark Core is the most explicit implementation. The RDD version exposes every map, key construction, accumulator merge, join, and sort. Its accumulator design is useful because it combines counts and sums before the shuffle and keeps the top-three cause logic deterministic. That explicitness also makes the code more verbose. Reproducing SQL ranking semantics, null handling, and top-three padding requires careful manual logic that Spark SQL expresses more directly.

Hive provides SQL-on-Hadoop style expressiveness, but it has the highest overhead in this project. The containerized Hive stack pays service, query startup, metastore, and export costs that are visible in the benchmark rows. Hive remains useful as a third technology and as a contrast with Spark, but the current local container setup does not demonstrate the strengths of a managed Hive service.

MapReduce is the most manual implementation. Hadoop Streaming makes the map and reduce boundaries explicit, and the Python reducers show the accumulator state required for averages, ranks, and top-three causes. The tradeoff is extra orchestration: prepared Parquet must be exported to canonical CSV, jobs must be run through Hadoop Streaming, and reducer output must be normalized back into the common CSV contract. This makes MapReduce valuable as a stretch comparison, but less ergonomic than Spark SQL for these analyses.

The delay-by-airport-month job is generally more expensive than the airline-airport ranking. It derives delay ranges, maps cancellation or delay causes, counts causes inside each airport-month-range group, and ranks causes to emit three positions. These steps increase shuffle and aggregation pressure. The ranking job also performs grouped aggregation and per-airport ranking, but after the first aggregation it works with compact airport-airline summaries.

Data preparation affects both correctness and benchmark fairness. Correctness is easier to audit because each technology reads the same typed Parquet schema instead of reimplementing raw CSV parsing. Benchmark fairness is stronger because all technologies operate on the same generated inputs. The limitation is that preparation cost is not included in the per-job execution-time tables; it is a separate mandatory pipeline stage documented in this report.

Small input sizes can distort runtime comparisons. JVM startup, Spark session startup, Docker container/service setup, Hive query startup, file listing, and query planning are fixed costs. Reads from prepared Parquet can also benefit from compact columnar layout and operating-system cache effects, while both analyses emit relatively small aggregate outputs compared with the input size. At 100k, these effects can dominate elapsed time; at larger sizes, throughput may improve even when raw seconds are flat or non-monotonic. The workflow therefore reports median duration and aggregate timing statistics when repeated runs are available, while preserving raw per-repetition CSV rows for audit.

The local, Docker, and EMR environments answer different questions. Local execution is the fastest in many cells because it avoids S3 reads, EMR step queueing, YARN container startup, and remote cluster coordination; it also runs near a warm local filesystem on a single tuned development machine. Docker standalone simulation adds service and container overhead and demonstrates a repeatable Spark standalone topology, but all workers still share one physical host. EMR is slower for several short jobs, especially Spark SQL delay aggregation, but it shows that the same analyses ran as managed Spark steps on real EC2-backed EMR clusters, read and wrote through S3, emitted logs and metrics to S3, and produced comparable outputs and benchmark rows. The larger EMR profile is intentionally treated as limited scalability evidence, not a universal speedup claim: adding one core node had lower Spark Core medians in the matched 1m and full cells, while Spark SQL was mixed because fixed Spark, S3, and scheduling costs remained a large share of these short analytical workloads.

10 Reproducibility And Validation

Generated large data, raw data, and full output directories are ignored by Git. The repository commits only small evaluator-facing evidence: report tables, figures, source code, documentation, and scripts.

Correctness validation is available through:

```
.\.venv\Scripts\python.exe scripts\validate_spark_sql_outputs.py
.\.venv\Scripts\python.exe scripts\validate_spark_core_outputs.py
.\.venv\Scripts\python.exe scripts\validate_hive_outputs.py
.\.venv\Scripts\python.exe scripts\validate_mapreduce_outputs.py
```

Spark Core, Hive, and MapReduce validators compare their outputs against Spark SQL, including output columns, row counts, key sets, numeric values within tolerance, top-three cause labels and counts, ranking order, and first-10 sample files.

11 Limitations

- The raw dataset and generated benchmark inputs are not committed to Git because they are large.
- The raw dataset does not include airline names, so analyses use `airline_code`.
- MapReduce depends on Docker-based Hadoop Streaming and is kept outside the default `make run-all-local` and `make benchmark-local` paths.
- The Docker Spark setup is a Docker standalone simulation on one physical machine, not a true distributed cluster.
- The Docker benchmark matrix includes `full`, `14m`, and `28m`, but it still runs on one physical host with shared Docker Desktop CPU, memory, and disk limits.
- The `1m_hc8` high-cardinality input is synthetic benchmark stress data. Its suffixed airport and airline keys are not real airports, airlines, or flight statistics.
- Hive is containerized locally and is not running on a Hadoop/YARN service.
- Docker worker-count variation is not included because the reliable Docker Compose topology has two named Spark workers.
- AWS evidence was produced inside AWS Academy Learner Lab, so service availability, IAM roles, instance availability, and the budget allowance were more constrained than a normal AWS account.
- EMR clusters were intentionally short-lived and benchmark-scoped. This is good for budget control but means the report does not measure long-running cluster warmup, autoscaling, or production scheduling behavior.
- S3 I/O is part of the EMR path. EMR timings therefore include object-store listing/read/write effects that local and Docker bind-mounted runs avoid.
- The Learner Lab budget was small, so the larger-cluster experiment has full repeated evidence for `1m` and `full`, a three-repetition larger `14m` run, and a single-run larger `28m` smoke/stress run rather than a complete sweep of every baseline cell.
- The largest AWS inputs still have limited repetitions: baseline `500k`, `3m`, and `14m` have two repetitions, larger `14m` has three repetitions, and larger `28m` has one smoke/stress repetition.
- Windows Spark runs require care around Java, Hadoop `winutils.exe`, and native file handling; the project includes compatibility helpers and Docker paths where needed.
- Benchmark results are hardware-dependent and should be interpreted as evidence for this controlled environment, not universal performance numbers.

12 Conclusion

The submitted artifacts show the main assignment work: a reproducible data preparation pipeline, Assignment Analyses 3.2 and 3.3, three technology implementations, first-10 result samples for each job and technology, benchmark tables, execution-time charts, and a critical discussion of expressiveness, implementation effort, efficiency, scalability, shuffle, aggregation, and data preparation effects. Spark SQL provides the

clearest reference logic, Spark Core demonstrates lower-level distributed transformation control, and Hive provides the required SQL-on-Hadoop comparison point. The completed MapReduce stretch adds a fourth validated implementation and shows the extra mechanics needed when using Hadoop Streaming directly.

The final submission also includes real cluster evidence: Spark SQL and Spark Core ran on Amazon EMR `emr-7.13.0` clusters in `us-east-1`, with S3 inputs, logs, metrics, fetched outputs, run manifests, step timing, and cost logs. This reduces the earlier weakness that the project only had local and Docker standalone simulation evidence, with the following limits: AWS Learner Lab budget and service constraints limited the number of repetitions, input sizes for the larger cluster, and breadth of cluster-size variation.

13 Evidence Appendix: First 10 Result Rows

The tables below embed the first 10 rows generated for Assignment Analysis 3.2 and Assignment Analysis 3.3 across the three required technologies. Wide outputs are split into paired narrow tables so the PDF remains readable. The source artifacts are stored under `report/tables/` and are regenerated by `make charts`. MapReduce stretch sample artifacts are generated with the same naming convention and summarized after the required-technology samples.

13.1 Spark SQL Samples

13.1.1 Delay Metrics

Origin	Mo.	Range	Flights	Avg dep.	Avg arr.
ABE	1	high	8	231.5	221.375
ABE	1	low	35	-6.286	-19.629
ABE	1	medium	4	32.75	31.25
ABE	2	high	3	324	335.667
ABE	2	low	40	-8.5	-23.575
ABE	2	medium	4	45.5	29.333
ABE	3	high	8	127	121.625
ABE	3	low	41	-4.756	-16.39
ABE	3	medium	3	32.333	12.667
ABE	4	high	6	158.333	150.333

13.1.2 Delay Top-Three Causes

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	1	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	1	low	unknown (34)	delay:nas (1)	none (0)
ABE	1	medium	delay:late_aircraft (2)	delay:carrier (1)	delay:nas (1)
ABE	2	high	delay:late_aircraft (1)	delay:nas (1)	delay:weather (1)
ABE	2	low	unknown (39)	delay:nas (1)	none (0)

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	2	medium	unknown (2)	delay:carrier (1)	delay:late_aircraft (1)
ABE	3	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	3	low	unknown (40)	delay:nas (1)	none (0)
ABE	3	medium	unknown (2)	delay:carrier (1)	none (0)
ABE	4	high	delay:carrier (4)	delay:late_aircraft (2)	none (0)

13.1.3 All-Positive Cause Companion

Origin	Mo.	Range	Rank	Cause	Count
ABE	1	cancelled_no_departure_delay	1	cancellation:B	11
ABE	1	low	1	delay:nas	21
ABE	1	low	2	delay:carrier	4
ABE	1	low	3	delay:late_aircraft	1
ABE	1	low	4	delay:weather	1
ABE	1	medium	1	delay:late_aircraft	16
ABE	1	medium	2	delay:nas	14
ABE	1	medium	3	delay:carrier	13
ABE	1	medium	4	delay:weather	3
ABE	1	medium	5	delay:security	1

13.1.4 Ranking Metrics

Origin	Airline	Flights	Avg dep.	Avg arr.	Cancel rate
ABE	9E	140	10.234	0.445	0.021
ABE	G4	260	12.463	4.337	0.012
ABE	OH	159	19.38	6.867	0.006
ABE	OO	40	25.297	14.389	0.075
ABI	MQ	249	6.664	4.405	0.008
ABQ	DL	218	1.383	-5.15	0.018
ABQ	OO	547	3.617	-0.855	0.004
ABQ	UA	273	4.584	-1.477	0.022
ABQ	MQ	127	7.134	5.575	0
ABQ	AS	67	7.522	1.881	0

13.1.5 Ranking Comparison

Origin	Airline	Airport avg dep.	Diff vs airport	Rank
ABE	9E	14.606	-4.373	1
ABE	G4	14.606	-2.143	2
ABE	OH	14.606	4.774	3
ABE	OO	14.606	10.691	4
ABI	MQ	6.664	0	1
ABQ	DL	8.792	-7.409	1
ABQ	OO	8.792	-5.176	2
ABQ	UA	8.792	-4.208	3
ABQ	MQ	8.792	-1.659	4
ABQ	AS	8.792	-1.27	5

13.2 Spark Core Samples

13.2.1 Delay Metrics

Origin	Mo.	Range	Flights	Avg dep.	Avg arr.
ABE	1	high	8	231.5	221.375
ABE	1	low	35	-6.286	-19.629

Origin	Mo.	Range	Flights	Avg dep.	Avg arr.
ABE	1	medium	4	32.75	31.25
ABE	2	high	3	324	335.667
ABE	2	low	40	-8.5	-23.575
ABE	2	medium	4	45.5	29.333
ABE	3	high	8	127	121.625
ABE	3	low	41	-4.756	-16.39
ABE	3	medium	3	32.333	12.667
ABE	4	high	6	158.333	150.333

13.2.2 Delay Top-Three Causes

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	1	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	1	low	unknown (34)	delay:nas (1)	none (0)
ABE	1	medium	delay:late_aircraft (2)	delay:carrier (1)	delay:nas (1)
ABE	2	high	delay:late_aircraft (1)	delay:nas (1)	delay:weather (1)
ABE	2	low	unknown (39)	delay:nas (1)	none (0)
ABE	2	medium	unknown (2)	delay:carrier (1)	delay:late_aircraft (1)
ABE	3	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	3	low	unknown (40)	delay:nas (1)	none (0)
ABE	3	medium	unknown (2)	delay:carrier (1)	none (0)
ABE	4	high	delay:carrier (4)	delay:late_aircraft (2)	none (0)

13.2.3 Ranking Metrics

Origin	Airline	Flights	Avg dep.	Avg arr.	Cancel rate
ABE	9E	140	10.234	0.445	0.021
ABE	G4	260	12.463	4.337	0.012
ABE	OH	159	19.38	6.867	0.006
ABE	OO	40	25.297	14.389	0.075
ABI	MQ	249	6.664	4.405	0.008
ABQ	DL	218	1.383	-5.15	0.018
ABQ	OO	547	3.617	-0.855	0.004
ABQ	UA	273	4.584	-1.477	0.022
ABQ	MQ	127	7.134	5.575	0
ABQ	AS	67	7.522	1.881	0

13.2.4 Ranking Comparison

Origin	Airline	Airport avg dep.	Diff vs airport	Rank
ABE	9E	14.606	-4.373	1
ABE	G4	14.606	-2.143	2
ABE	OH	14.606	4.774	3
ABE	OO	14.606	10.691	4
ABI	MQ	6.664	0	1
ABQ	DL	8.792	-7.409	1
ABQ	OO	8.792	-5.176	2

Origin	Airline	Airport avg dep.	Diff vs airport	Rank
ABQ	UA	8.792	-4.208	3
ABQ	MQ	8.792	-1.659	4
ABQ	AS	8.792	-1.27	5

13.3 Hive Samples

13.3.1 Delay Metrics

Origin	Mo.	Range	Flights	Avg dep.	Avg arr.
ABE	1	high	8	231.5	221.375
ABE	1	low	35	-6.286	-19.629
ABE	1	medium	4	32.75	31.25
ABE	2	high	3	324	335.667
ABE	2	low	40	-8.5	-23.575
ABE	2	medium	4	45.5	29.333
ABE	3	high	8	127	121.625
ABE	3	low	41	-4.756	-16.39
ABE	3	medium	3	32.333	12.667
ABE	4	high	6	158.333	150.333

13.3.2 Delay Top-Three Causes

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	1	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	1	low	unknown (34)	delay:nas (1)	none (0)
ABE	1	medium	delay:late_aircraft (2)	delay:carrier (1)	delay:nas (1)
ABE	2	high	delay:late_aircraft (1)	delay:nas (1)	delay:weather (1)
ABE	2	low	unknown (39)	delay:nas (1)	none (0)

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	2	medium	unknown (2)	delay:carrier (1)	delay:late_aircraft (1)
ABE	3	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	3	low	unknown (40)	delay:nas (1)	none (0)
ABE	3	medium	unknown (2)	delay:carrier (1)	none (0)
ABE	4	high	delay:carrier (4)	delay:late_aircraft (2)	none (0)

13.3.3 Ranking Metrics

Origin	Airline	Flights	Avg dep.	Avg arr.	Cancel rate
ABE	9E	140	10.234	0.445	0.021
ABE	G4	260	12.463	4.337	0.012
ABE	OH	159	19.38	6.867	0.006
ABE	OO	40	25.297	14.389	0.075
ABI	MQ	249	6.664	4.405	0.008
ABQ	DL	218	1.383	-5.15	0.018
ABQ	OO	547	3.617	-0.855	0.004

Origin	Airline	Flights	Avg dep.	Avg arr.	Cancel rate
ABQ	UA	273	4.584	-1.477	0.022
ABQ	MQ	127	7.134	5.575	0
ABQ	AS	67	7.522	1.881	0

13.3.4 Ranking Comparison

Origin	Airline	Airport avg dep.	Diff vs airport	Rank
ABE	9E	14.606	-4.373	1
ABE	G4	14.606	-2.143	2
ABE	OH	14.606	4.774	3
ABE	OO	14.606	10.691	4
ABI	MQ	6.664	0	1
ABQ	DL	8.792	-7.409	1
ABQ	OO	8.792	-5.176	2
ABQ	UA	8.792	-4.208	3
ABQ	MQ	8.792	-1.659	4
ABQ	AS	8.792	-1.27	5

13.4 MapReduce Stretch Samples

The MapReduce stretch implements both selected analyses with Hadoop Streaming. It is optional evidence after the required Spark SQL, Spark Core, and Hive samples above. The validated full prepared-data run produced 11,902 delay rows and 1,738 ranking rows, matching the Spark SQL reference on keys, numeric fields, rank values, and top-three cause labels/counts.

Artifact	Path
Runtime metrics	outputs/mapreduce/runtime_metrics.json
Delay sample	report/tables/first_10_mapreduce_delay_by_airport_month.md
All-positive cause sample	report/tables/first_10_mapreduce_delay_by_airport_month_all_causes.md
Ranking sample	report/tables/first_10_mapreduce_airline_airport_ranking.md
Validator	scripts/validate_mapreduce_outputs.py
Runtime documentation	docs/mapreduce_analyses.md

The MapReduce stretch benchmark completed 100k, 500k, 1m, 3m, and full inputs with three repetitions per input/job using isolated benchmark outputs. These rows are included in generated report tables when present, but missing MapReduce cells are not added to the required Spark/Hive/Spark Core benchmark status matrix.